

Universidad Autónoma del Estado de México

Ingeniería de Requerimientos

Técnicas, modelado, especificación, validación y gestión

Material didáctico

Conceptos fundamentales, ejemplos aplicados y buenas prácticas

Presentación

La Ingeniería de Requerimientos es la fase del desarrollo de software donde se descubre, analiza, documenta y administra qué debe hacer un sistema. Equivocarse aquí cuesta mucho más caro que equivocarse en el código: un error de requerimientos detectado durante el mantenimiento puede llegar a costar hasta 100 veces más que si se hubiera detectado al inicio.

Este documento explica de manera clara y con ejemplos prácticos las técnicas y procesos centrales de la disciplina. Cada sección incluye: (a) el concepto clave, (b) cómo se aplica, (c) un ejemplo concreto y (d) buenas prácticas para que lo apliques en tus proyectos.

EN LA PRÁCTICA — ¿Por qué importa para un ingeniero en computación?

Un buen requerimiento es la frontera entre **construir lo que el cliente necesita** y **construir lo que el desarrollador imaginó**. La primera situación genera software exitoso; la segunda genera retrabajo, frustración y proyectos cancelados.

3.2 Técnicas de recolección de requerimientos

La recolección (también llamada elicitación) es el proceso de obtener información de los interesados — usuarios, clientes, expertos del dominio— sobre lo que el sistema debe hacer. No existe una técnica única que funcione para todo: cada situación pide una combinación distinta.

CONCEPTO CLAVE — ¿Qué es recolectar requerimientos?

Es la actividad de descubrir, mediante interacción con personas y documentos, las necesidades reales que debe satisfacer el software. El reto no es solo escuchar: es interpretar, contrastar y completar lo que los interesados a veces no saben expresar.

Ojo: los usuarios rara vez te dicen literalmente lo que necesitan. Suelen describir síntomas ("el sistema actual es lento") o soluciones imaginadas ("quiero un botón rojo aquí"). Tu trabajo es llegar al problema real.

Entrevistas

Conversaciones uno a uno (o en grupos pequeños) con interesados clave. Son la técnica más común porque permite profundizar, aclarar dudas en el momento y captar matices.

Tipos de entrevista

- **Estructurada:** sigues un guion fijo de preguntas. Útil cuando entrevistas a muchas personas y necesitas comparar respuestas.
- **Semiestructurada:** tienes una guía pero permites desviarte si surge algo interesante. Es la más usada en la práctica.
- **Abierta:** solo planteas un tema y dejas que el entrevistado hable. Útil al inicio del proyecto para descubrir lo desconocido.

EJEMPLO — Entrevista a la coordinadora de biblioteca

Proyecto: Sistema de préstamo de libros para una biblioteca universitaria.

Pregunta abierta: "Cuénteme cómo es un día normal en el mostrador de préstamos."

Respuesta reveladora: "Lo más pesado es a las 8 a.m. cuando llegan los alumnos a devolver libros del día anterior antes de su clase de 9. Si el sistema se cae, formamos colas de 30 personas."

Requerimiento descubierto: el sistema debe tolerar picos de 100 devoluciones en 15 minutos y tener un modo offline para registrar devoluciones cuando la red falle. *Esto no habría salido en un cuestionario cerrado.*

Buenas prácticas para entrevistar

- Prepara las preguntas, pero no las leas como interrogatorio.
- Pregunta "¿por qué?" varias veces para llegar a la causa raíz (técnica de los 5 porqués).
- Toma notas y, si el entrevistado lo permite, graba con consentimiento explícito.
- Cierra cada entrevista con un resumen verbal: "Entonces, lo que entendí es... ¿es correcto?"
- Nunca entrevistes solo a los jefes: los usuarios operativos conocen los problemas reales.

Cuestionarios

Conjunto estructurado de preguntas que se aplica a muchas personas. Útil cuando necesitas información de decenas o cientos de usuarios y no es viable entrevistar a todos.

Aspecto	Cuándo usar cuestionarios	Cuándo NO usarlos
Tamaño de la audiencia	Más de 20 personas	Pocas personas accesibles
Tipo de información	Datos cuantificables, preferencias, frecuencias	Conceptos complejos o sentimientos profundos
Madurez del problema	Ya conoces las dimensiones del problema	Estás explorando lo desconocido

Aspecto	Cuándo usar cuestionarios	Cuándo NO usarlos
Ejemplo de uso	Encuesta a 200 alumnos sobre qué módulos del SIGA usan más	Definir flujo del proceso de titulación

✓ EN LA PRÁCTICA — Trampas frecuentes al diseñar cuestionarios

- Preguntas con doble intención ("¿el sistema es rápido y fácil de usar?") — si responde no, no sabes a qué se refiere.
- Preguntas que sugieren la respuesta ("¿no le parece que sería útil...?").
- Demasiadas preguntas abiertas: la gente abandona el cuestionario.
- Escalas inconsistentes (mezclar 1-5, 1-10 y sí/no sin razón).

Talleres (workshops)

Sesiones grupales facilitadas donde varios interesados trabajan juntos para definir requerimientos. La variante más conocida es JAD (Joint Application Design) y, en entornos ágiles, los talleres de story mapping o event storming.

Ventajas de los talleres

- Permiten resolver desacuerdos entre áreas en el momento (no por correo durante semanas).
- Generan compromiso: lo que se acuerda en sala es difícil de negar después.
- Sacan a la luz dependencias e interacciones entre áreas que en entrevistas individuales no se ven.

🔗 EJEMPLO — Taller para un sistema de control escolar

Asistentes: control escolar, docentes, alumnos representantes, sistemas (DTIC), dirección.

Dinámica: se dibuja en un pizarrón el flujo "alumno solicita kardex". Cada participante señala dónde se atora hoy.

Hallazgo: control escolar pensaba que el cuello de botella era la impresión; los alumnos en realidad sufren porque no saben en qué ventanilla preguntar. El requerimiento resultante no es "impresoras más rápidas" sino "notificación al alumno con el estatus del trámite".

Análisis de documentos

Revisión de documentación existente: manuales del sistema actual, formatos en papel, normativas, reglamentos, reportes, contratos. Es una técnica subestimada pero muy productiva, especialmente cuando se reemplaza un sistema legado o se automatiza un proceso manual.

Qué buscar en los documentos

- Reglas de negocio explícitas ("un préstamo no puede exceder 14 días").
- Campos de datos obligatorios y opcionales en formatos actuales.
- Flujos de aprobación y firmas requeridas.
- Excepciones documentadas ("en caso de...").
- Restricciones legales o normativas que el sistema debe cumplir.

EJEMPLO — Lo que un reglamento te ahorra preguntar

Contexto: automatizar el registro de calificaciones.

Documento clave: Reglamento de Facultad.

Reglas extraídas sin entrevistar a nadie: (1) las calificaciones se capturan en escala 0–10, (2) el docente tiene 10 días hábiles después del examen para capturar, (3) una vez firmada el acta, solo el secretario académico puede modificarla.

Ventaja: llegas a las entrevistas con un borrador de reglas y solo confirmas o detectas excepciones, en lugar de descubrirlas desde cero.

Cómo elegir y combinar técnicas

En la práctica nunca se usa una sola técnica. Una secuencia típica al inicio de un proyecto es:

1. Análisis de documentos para entender el contexto sin desgastar a los usuarios.
2. Entrevistas con interesados clave para profundizar.
3. Taller para integrar visiones y resolver conflictos.
4. Cuestionario para validar prioridades con la población amplia.

3.3 Análisis y modelado de requerimientos mediante UML

Después de recolectar la información cruda, debemos organizarla, encontrar contradicciones y representarla de forma que tanto el cliente como el equipo de desarrollo entiendan lo mismo. UML (Unified Modeling Language) es el lenguaje gráfico estándar para hacerlo.

CONCEPTO CLAVE — ¿Por qué modelar y no solo escribir?

El texto en lenguaje natural es ambiguo: la frase "el sistema notifica al usuario" no dice cuándo, cómo, ni a quién exactamente. Un diagrama obliga a precisar relaciones, actores y secuencias.

Modelar también ayuda a **detectar huecos**: cuando dibujas un diagrama de casos de uso, rápidamente notas si un actor importante quedó fuera o si falta un caso de uso para una excepción.

Diagramas UML más usados en análisis de requerimientos

Diagrama	Qué representa	Cuándo usarlo
Casos de uso	Funcionalidad del sistema vista desde los actores externos	Para acordar el alcance con el cliente
Actividades	Flujo paso a paso de un proceso, con decisiones y paralelismo	Para modelar procesos de negocio o flujos complejos
Secuencia	Interacción cronológica entre objetos para realizar un caso de uso	Para detallar cómo se cumple un caso de uso
Clases (de análisis)	Conceptos del dominio y sus relaciones	Para modelar el vocabulario y los datos del negocio
Estados	Comportamiento de un objeto a lo largo de su ciclo de vida	Para entidades con muchos estados (pedido, préstamo)

Diagrama de casos de uso: el más cercano al cliente

Es el diagrama más útil para discutir con un cliente no técnico. Tiene tres elementos esenciales:

- **Actor:** rol externo que interactúa con el sistema (usuario, otro sistema, tiempo). Se dibuja como un "monito".
- **Caso de uso:** objetivo que el actor logra usando el sistema. Se dibuja como un óvalo y se nombra con verbo + objeto ("Registrar préstamo").
- **Relación:** líneas que conectan actores con casos de uso. También hay relaciones especiales: *include* (un caso de uso siempre incluye otro), *extend* (lo extiende solo en ciertas condiciones).

EJEMPLO — Casos de uso para una biblioteca

Actores: Alumno, Bibliotecario, Sistema de Pagos (externo).

Casos de uso del Alumno: Buscar libro, Reservar libro, Consultar mis préstamos, Pagar multa.

Casos de uso del Bibliotecario: Registrar préstamo, Registrar devolución, Dar de alta libro, Generar reporte de morosos.

Relación include: "Registrar préstamo" incluye "Verificar identidad del alumno" (siempre se verifica).

Relación extend: "Registrar devolución" se extiende con "Calcular multa" (solo si hay retraso).

Plantilla de descripción textual de un caso de uso

El diagrama da una visión panorámica, pero cada caso de uso necesita una descripción textual detallada que sirva como contrato funcional:

Campo	Contenido
Nombre	Verbo + objeto. Ej: Registrar préstamo.
Actor principal	Quien inicia el caso de uso.
Precondición	Lo que debe ser verdadero antes de iniciar.
Postcondición	El estado del sistema cuando termina exitosamente.
Flujo principal	Pasos numerados del escenario exitoso.
Flujos alternativos	Variantes y excepciones (libro no disponible, etc.).
Requisitos no funcionales	Tiempo de respuesta, seguridad, etc., aplicables al caso.

EJEMPLO — Caso de uso: Registrar préstamo (forma textual)

Actor principal: Bibliotecario.

Precondición: El bibliotecario está autenticado y el libro está dado de alta en el catálogo.

Flujo principal:

- 1. El bibliotecario escanea el código del libro.
- 2. El sistema muestra los datos del libro y verifica disponibilidad.

- 3. El bibliotecario escanea la credencial del alumno.
- 4. El sistema verifica que el alumno no tenga multas pendientes ni préstamos vencidos.
- 5. El sistema registra el préstamo con fecha de devolución a 14 días.
- 6. El sistema imprime el comprobante.

Flujo alternativo 4a: el alumno tiene multas. El sistema muestra el monto y bloquea el préstamo hasta que se pague.

Postcondición: el libro queda marcado como prestado y el alumno suma un préstamo activo.

3.4 Especificación de requerimientos

Especificar significa escribir los requerimientos de manera precisa, completa y verificable, en un documento que servirá como referencia durante todo el ciclo de vida del software. El documento típico se llama SRS (Software Requirements Specification) y su estructura está sugerida por el estándar IEEE 830.

CONCEPTO CLAVE — Tipos de requerimientos

Funcionales: qué debe hacer el sistema. Ejemplo: "el sistema debe permitir registrar un préstamo de libro".

No funcionales: cómo debe comportarse el sistema. Ejemplo: "el registro de préstamo debe completarse en menos de 2 segundos con 100 usuarios concurrentes".

De dominio: restricciones que vienen del campo de aplicación. Ejemplo: "se aplican las disposiciones del Reglamento de Bibliotecas Universitarias".

Buenas prácticas para redactar requerimientos

Plantilla recomendada (formato EARS / Easy Approach to Requirements Syntax)

Es una de las plantillas más usadas en la industria por su simplicidad. Tiene cinco variantes:

- **Ubicua:** "El sistema DEBE <comportamiento>". Aplica siempre.
- **Activada por evento:** "CUANDO <evento>, el sistema DEBE <comportamiento>".
- **No deseada:** "SI <condición no deseada>, ENTONCES el sistema DEBE <reacción>".
- **Estado:** "MIENTRAS <estado>, el sistema DEBE <comportamiento>".
- **Opcional:** "DONDE <característica habilitada>, el sistema DEBE <comportamiento>".

EJEMPLO — Requerimientos bien redactados (formato EARS)

RF-01 (ubicua): El sistema DEBE permitir al bibliotecario registrar préstamos de libros.

RF-02 (evento): CUANDO un préstamo cumpla 14 días sin devolución, el sistema DEBE enviar un correo de recordatorio al alumno.

RF-03 (no deseada): SI el alumno tiene multas pendientes, ENTONCES el sistema DEBE bloquear nuevos préstamos.

RNF-01: El sistema DEBE responder a cualquier operación de consulta en menos de 2 segundos con 100 usuarios concurrentes.

Características de un buen requerimiento

Característica	Significa que...	Ejemplo negativo
Completo	No falta información necesaria para implementarlo	"El sistema debe notificar." ¿A quién? ¿Cuándo? ¿Cómo?
No ambiguo	Solo admite una interpretación	"El sistema debe ser rápido." ¿Qué es rápido?
Verificable	Se puede probar si se cumple o no	"El sistema debe ser intuitivo." ¿Cómo se mide?
Consistente	No contradice otros requerimientos	Un requisito dice retención de 5 años, otro dice 3
Trazable	Tiene un identificador único y se puede rastrear	Lista sin numeración ni referencias
Necesario	Si se elimina, se pierde valor real	"El botón debe ser azul." (decisión de diseño, no de negocio)
Factible	Es posible implementarlo con tiempo y recursos disponibles	"Predecir el clima con 100% de exactitud."

Verificación de la especificación

Verificar es comprobar que el documento de requerimientos esté bien construido (sin errores, ambigüedades o inconsistencias). Es una revisión interna antes de mostrarlo al cliente. Las técnicas habituales son:

- **Inspecciones formales:** reuniones donde un equipo revisa el documento siguiendo una lista de verificación.
- **Listas de chequeo (checklists):** preguntas predefinidas ("¿cada requerimiento es verificable?", "¿hay términos no definidos?").
- **Lectura por roles:** cada revisor lee el documento desde la perspectiva de un actor distinto.

Elicitación (relación con la recolección)

El término elicitación abarca todo el proceso de extraer y refinar requerimientos, no solo la recolección inicial. Incluye recolectar, analizar y negociar. En el SRS, esta sección documenta cómo se obtuvieron los requerimientos, qué fuentes se usaron y qué supuestos se hicieron, para que cualquier lector pueda evaluar la credibilidad del documento.

☑ **EN LA PRÁCTICA — Antipatrones que debes evitar en una especificación**

- **Verbos vagos:** "manejar", "procesar", "gestionar". Sustituye por verbos concretos: "registrar", "validar", "calcular".
- **Adjetivos sin métrica:** "amigable", "robusto", "flexible". Si no puedes medirlo, no puedes verificarlo.
- **Mezclar el qué con el cómo:** decir "usar MySQL" en un requerimiento funcional limita opciones del equipo de diseño.
- **Requerimientos compuestos:** "el sistema debe registrar el préstamo y enviar un correo." Sepáralos: uno se puede cumplir y el otro no.

3.5 Métodos de revisión y validación (aseguramiento de la calidad)

Una vez que el documento de requerimientos está redactado y verificado internamente, debe validarse con el cliente y los usuarios. La diferencia es importante:

CONCEPTO CLAVE — Verificación vs. Validación

Verificación: ¿estamos construyendo el producto *correctamente*? Revisa que el documento esté bien hecho (sin errores formales).

Validación: ¿estamos construyendo *el producto correcto*? Revisa que los requerimientos reflejen lo que el cliente realmente necesita.

Técnicas de validación de requerimientos

1. Revisión formal con el cliente

Se entrega el SRS al cliente y se realiza una reunión de revisión sección por sección. Es la técnica básica, pero tiene un problema: los clientes no técnicos no siempre detectan ambigüedades en un texto largo.

2. Prototipado

Construir una versión preliminar (a veces solo maquetas o mockups, a veces un prototipo funcional limitado) que el cliente pueda "tocar". Permite validar requerimientos que en papel suenan bien pero en uso revelan problemas.

EJEMPLO — Prototipado en un sistema de pedidos

Requerimiento en papel: "El usuario puede seleccionar productos y agregarlos al carrito."

El cliente lo aprueba.

Se construye un prototipo en Figma. El usuario nota que necesita ver imágenes grandes del producto, comparar dos productos lado a lado y guardar productos para después sin agregarlos al carrito.

Tres requerimientos nuevos descubiertos antes de programar nada.

3. Casos de prueba derivados

Por cada requerimiento, escribir uno o varios casos de prueba antes de implementar. Si no logras escribir un caso de prueba para un requerimiento, ese requerimiento no es verificable y debe reescribirse.

4. Validación con escenarios y casos de uso

Recorrer cada caso de uso con el cliente, contando la historia paso a paso: "imagine que un alumno llega al mostrador...". Las historias narradas revelan vacíos que las listas de requerimientos no.

5. Walkthroughs y inspecciones

Reuniones estructuradas donde el autor del documento guía a los revisores a través del contenido. En una inspección formal hay roles definidos (moderador, lector, registrador, autor) y métricas de defectos encontrados.

Criterios de aceptación

Cada requerimiento debe acompañarse de criterios de aceptación: condiciones concretas y medibles que indican cuándo se considera cumplido. Es la base para la prueba final con el cliente.

EJEMPLO — Criterios de aceptación para RF-03

Requerimiento: Si el alumno tiene multas pendientes, el sistema debe bloquear nuevos préstamos.

Criterios de aceptación:

- Dado un alumno con multa de \$50 pendiente, cuando el bibliotecario intente registrar un préstamo, entonces el sistema mostrará el mensaje "Préstamo bloqueado: multa pendiente de \$50" y no creará el registro.
- Dado un alumno sin multas, cuando el bibliotecario registre un préstamo, entonces el sistema lo crea normalmente.
- Dado un alumno que acaba de pagar su multa, cuando el bibliotecario intente el préstamo en los siguientes 5 segundos, entonces el sistema lo permite (no usar caché obsoleto).

3.6 Gestión de requerimientos

Los requerimientos no son estáticos: cambian a lo largo del proyecto porque el negocio cambia, los usuarios descubren nuevas necesidades, surgen restricciones técnicas o legales, o simplemente porque al ver el sistema en construcción se dan cuenta de lo que realmente querían. Gestionar requerimientos es controlar este flujo de cambios sin perder el rumbo del proyecto.

CONCEPTO CLAVE — ¿Qué incluye la gestión de requerimientos?

- Identificar y clasificar cada requerimiento con un ID único.
- Priorizar (qué es indispensable, qué es deseable).
- Asignar requerimientos a iteraciones o entregas.
- Rastrear el estado de cada requerimiento (propuesto, aprobado, implementado, verificado).
- Mantener la trazabilidad con código, pruebas y documentación.
- Procesar cambios de manera controlada.

Priorización de requerimientos

No todos los requerimientos tienen el mismo peso. Las técnicas más usadas para priorizarlos son:

Técnica	Cómo funciona	Cuándo usarla
MoSCoW	Clasificar en Must (debe), Should (debería), Could (podría), Won't (no esta vez)	Para acordar alcance con cliente al inicio
Valor vs. Esfuerzo	Estimar valor de negocio y esfuerzo de implementación; priorizar alto valor / bajo esfuerzo	Para decidir el orden de implementación
100 puntos	Cada interesado distribuye 100 puntos entre los requerimientos	Para equilibrar voces de varios interesados
Kano	Clasificar en básicos, lineales y atractivos	Para productos comerciales con competencia

EJEMPLO — Priorización MoSCoW para un sistema de biblioteca (MVP)

Must: registrar préstamo, registrar devolución, autenticar usuarios, consultar catálogo.

Should: calcular multas automáticamente, enviar recordatorios por correo.

Could: reservar libros desde el portal, recomendaciones según historial.

Won't (esta versión): integración con sistemas de otras universidades, app móvil nativa.

Estados del ciclo de vida de un requerimiento

- **Propuesto:** alguien lo sugirió, todavía no se aprueba.
- **Aprobado:** fue revisado y aceptado para la línea base.
- **En desarrollo:** está siendo implementado.
- **Implementado:** el código está escrito y compila.
- **Verificado:** pasó las pruebas internas.
- **Validado:** el cliente aceptó que está bien implementado.
- **Rechazado o diferido:** se decidió no implementarlo o postergarlo.

3.7 Control de cambios en los requerimientos (trazabilidad y herramientas)

Los cambios son inevitables, pero deben ser controlados. Un cambio mal gestionado es la primera causa de retraso y sobrecosto en proyectos de software. El control de cambios establece un proceso formal para evaluar, aprobar y registrar cada modificación.

Proceso típico de control de cambios

5. Solicitud de cambio (Change Request): cualquier interesado puede solicitar un cambio. Se registra con descripción, justificación y solicitante.
6. Análisis de impacto: el equipo evalúa qué requerimientos, módulos, pruebas y documentos se ven afectados, y estima el esfuerzo adicional.
7. Decisión: el Comité de Control de Cambios (CCB) aprueba, rechaza o difiere el cambio. En proyectos pequeños puede ser solo el líder del proyecto y el cliente.
8. Implementación: se actualizan los documentos (SRS, casos de uso, pruebas) y se realiza el cambio en el código.
9. Verificación y cierre: se verifica que el cambio se aplicó correctamente y se cierra el Change Request.

EJEMPLO — Una solicitud de cambio bien documentada

ID: CR-2026-014

Solicitante: Coordinadora de biblioteca

Descripción: Permitir préstamos de 21 días en lugar de 14 para alumnos de posgrado.

Justificación: Los alumnos de posgrado leen libros más extensos y 14 días es insuficiente.

Requerimientos afectados: RF-01, RF-02, RNF-03.

Impacto estimado: 2 días de desarrollo, 1 día de pruebas, actualización del SRS y de 4 casos de prueba.

Decisión CCB (2026-05-12): Aprobado para release 2.1.

Trazabilidad de requerimientos

La trazabilidad es la capacidad de seguir un requerimiento desde su origen hasta su implementación y prueba, y de vuelta. Es esencial para responder preguntas como: "si quito este requerimiento, ¿qué pruebas dejan de tener sentido?" o "¿de dónde salió esta función del código?".

CONCEPTO CLAVE — Tipos de trazabilidad

Hacia atrás (backward): del requerimiento hacia su origen (quién lo solicitó, en qué reunión, qué documento del cliente lo justifica).

Hacia adelante (forward): del requerimiento hacia los artefactos derivados (diseño, código, pruebas).

Horizontal: entre requerimientos del mismo nivel (qué requerimientos dependen entre sí).

Matriz de trazabilidad

La forma más común de documentar la trazabilidad. Es una tabla que cruza requerimientos con casos de uso, módulos de código y casos de prueba.

Requerimiento	Caso de uso	Módulo de código	Caso de prueba
RF-01 Registrar préstamo	CU-03	PrestamoService.java	TP-15, TP-16
RF-02 Recordatorio 14 días	CU-07	RecordatorioScheduler.java	TP-22
RF-03 Bloquear multas	CU-03 (alt 4a)	ValidacionAlumno.java	TP-17, TP-18
RNF-01 Respuesta <2s	(transversal)	(varios)	TP-30 (carga)

Herramientas de gestión de requerimientos

Para proyectos pequeños puedes usar una hoja de cálculo o un documento bien estructurado. Para proyectos medianos y grandes existen herramientas especializadas:

Categoría	Herramientas representativas	Fuerte en
Trazabilidad formal	IBM DOORS, Jama Connect, Polarion	Industrias reguladas (médica, aeroespacial, automotriz)
Ágil / Backlog	Jira, Azure DevOps, GitHub Issues, Linear	Equipos Scrum/Kanban con historias de usuario
Modelado UML	Enterprise Architect, Visual Paradigm, StarUML, draw.io	Diagramas de casos de uso, clases, secuencia
Documentos colaborativos	Confluence, Notion, SharePoint	SRS, actas, espacios de trabajo del equipo
Prototipado	Figma, Balsamiq, Adobe XD	Validación temprana con usuarios

 EN LA PRÁCTICA — Recomendaciones para elegir herramienta

- Empieza por lo más sencillo: una buena hoja de cálculo con identificadores y estados ya gestiona requerimientos para proyectos académicos.
- Si el equipo trabaja en Scrum, mantén los requerimientos en el mismo backlog donde planeas (Jira, Azure DevOps): evita la doble captura.
- La herramienta no sustituye al proceso. Si no tienes claro qué es un cambio aprobado, ninguna herramienta te ayudará.
- Garantiza que cada requerimiento tenga ID, estado, prioridad, fuente y al menos un caso de prueba asociado.

Síntesis

La Ingeniería de Requerimientos no es una fase aislada que termina antes del diseño: es un trabajo continuo que abarca recolectar, modelar, especificar, validar, gestionar y controlar cambios. Su producto principal —el SRS— es el contrato técnico-funcional sobre el que se construye todo el software.

Mapa mental de los temas

Subtema	Pregunta que responde	Producto principal
3.2 Recolección	¿Qué necesita el usuario?	Notas, transcripciones, hallazgos
3.3 Análisis y UML	¿Cómo se organiza lo que sabemos?	Diagramas de casos de uso, actividades, clases
3.4 Especificación	¿Cómo lo escribimos sin ambigüedad?	Documento SRS (IEEE 830)
3.5 Validación	¿Es lo que el cliente realmente quiere?	Acta de aceptación, prototipos validados
3.6 Gestión	¿Cómo lo administramos en el tiempo?	Backlog priorizado, matriz de estados
3.7 Control de cambios	¿Cómo controlamos lo que cambia?	Change Requests, matriz de trazabilidad